

AI FAQ ASSISTANT API

FINAL PROJECT DOCUMENTATION

Secure • Intelligent • RESTful • AI-Powered FAQ Management

PROJECT INFORMATION	DETAILS
Project Title	AI FAQ Assistant API
Team ID	SWTID-2026-8114
Team Size	4 Members
Team Leader	SANJU SHREE R
Team Members	KRITHIKA N • KIRTHIGA R • S. DEEPIKA
Technology Domain	AI-Augmented Backend Development
Project Type	REST API / Backend Application

Prepared for SmartBridge / SkillWallet Project Submission

1. Introduction

Project Title: AI FAQ Assistant API

Team: SANJU SHREE R (Team Leader), KRITHIKA N, KIRTHIGA R, and S. DEEPIKA

Team ID: SWTID-2026-8114

The AI FAQ Assistant API is a backend system designed to simplify FAQ creation, management, search, authentication, and AI-assisted content generation. It exposes RESTful endpoints that can be consumed by web applications, mobile applications, or other client systems.

2. Project Overview

The main objective is to provide a secure and scalable backend for customer-support and knowledge-base workflows. The system combines conventional FAQ management with Google Gemini-powered content generation.

Core Capabilities

- JWT-based authentication and authorization for protected resources.
- AI-assisted generation of FAQ questions, answers, and categories from a topic.
- Keyword-based FAQ search for quick information retrieval.
- Create, read, update, and delete operations for FAQ records.
- Category-based organization of knowledge content.
- bcrypt-based password hashing before persistence.
- Mongoose schema validation and centralized error handling.
- RESTful API design using Node.js, Express.js, MongoDB, and Mongoose.

3. System Architecture

Layer	Responsibility	Technology
Client / API Tool	Sends HTTP requests and receives JSON responses.	Postman / Thunder Client
Routing Layer	Maps HTTP methods and paths to application operations.	Express Router
Controller Layer	Handles request validation and business logic.	Node.js / Express.js
Service / AI Layer	Processes AI-assisted FAQ generation.	Google Gemini SDK
Model Layer	Defines schemas, validation, references, and persistence logic.	Mongoose
Database Layer	Stores users and FAQ records.	MongoDB
Security Middleware	Verifies JWT tokens and controls protected access.	JWT / bcrypt

Architecture Pattern: MVC-inspired layered backend architecture. Because this is a headless REST API, the routing layer serves as the API-facing presentation layer rather than a traditional HTML view.

4. Software and Hardware Requirements

Category	Requirement
Operating System	Windows 10/11, macOS, or Linux
Runtime	Node.js v18+ and npm v9+

Category	Requirement
Backend	Express.js
Database	MongoDB with Mongoose ODM
AI Integration	Google Gemini SDK (@google/genai)
Security	jsonwebtoken and bcrypt
Configuration	dotenv and CORS
API Testing	Postman or Thunder Client
Recommended Hardware	Intel Core i5 (8th Gen+) / AMD Ryzen 5 or equivalent; 8 GB RAM minimum; 16 GB recommended
Storage	At least 1 GB free disk space

5. Database Design

5.1 User Collection

Field	Type	Constraint / Purpose
_id	ObjectId	Primary identifier
name	String	Required
email	String	Required and unique
password	String	Required; stored as a bcrypt hash
role	String	Defaults to user
createdAt	Date	Automatic timestamp
updatedAt	Date	Automatic timestamp

5.2 FAQ Collection

Field	Type	Constraint / Purpose
_id	ObjectId	Primary identifier
question	String	Required
answer	String	Required
category	String	Required
createdBy	ObjectId	Reference to User
createdAt	Date	Automatic timestamp
updatedAt	Date	Automatic timestamp

5.3 Logical AI Generation Data

Field	Purpose
topic	Input topic supplied for generation

Field	Purpose
generatedQuestion	AI-generated FAQ question
generatedAnswer	AI-generated FAQ answer
generatedCategory	AI-generated or assigned category
generatedAt	Generation timestamp

5.4 Key Relationships

- **User → FAQ (One-to-Many):** one user can create multiple FAQ records.
- **Category → FAQ (One-to-Many):** one category can contain multiple FAQ records.
- **User → AI-assisted FAQ (One-to-Many):** one user can generate multiple AI-assisted FAQ entries.

6. Roles and Responsibilities

Role	Primary Responsibilities
Administrator	Manage users and FAQ content, monitor AI-generated content, manage categories, and perform system-level operations.
Content Creator	Create, edit, and delete owned FAQs; generate AI-assisted FAQs; and organize content.
Authenticated User	View FAQs, search content, generate permitted AI-assisted answers, and access permitted profile information.
Public User	View published FAQs and search available content without access to protected modification operations.

7. Setup and Installation

Prerequisites

- Node.js v18+
- MongoDB
- npm
- Git
- Postman or Thunder Client

Installation Procedure

1. Create the project folder and open it in Visual Studio Code.
2. Initialize the Node.js project using `npm init -y`.
3. Install the required dependencies using `npm install express mongoose dotenv cors jsonwebtoken bcrypt @google/genai`.
4. Create the `.env` file and configure the MongoDB URI, JWT secret, and Gemini API key.
5. Start the application using the project's configured `npm start/development` command.
6. Verify the server and database connection before testing API endpoints.

8. Project Structure

```
AI-FAQ-Assistant-API/  
  ■■■ src/  
  ■ ■■■ server.js
```

```
■ ■ ■ ■ app.js
■ ■ ■ ■ middleware/
■ ■ ■ ■ authMiddleware.js
■ ■ ■ ■ errorMiddleware.js
■ ■ ■ ■ models/
■ ■ ■ ■ User.js
■ ■ ■ ■ FAQ.js
■ ■ ■ ■ routes/
■ ■ ■ ■ authRoutes.js
■ ■ ■ ■ faqRoutes.js
■ ■ ■ ■ aiRoutes.js
■ ■ ■ ■ controllers/
■ ■ ■ ■ .env
■ ■ ■ ■ package.json
■ ■ ■ ■ README.md
```

9. API Documentation

Method	Endpoint	Purpose	Access
POST	/api/auth/register	Register a new user	Public
POST	/api/auth/login	Authenticate user and return JWT	Public
POST	/api/faqs	Create an FAQ	Protected
GET	/api/faqs	Retrieve FAQ records / published content	Public
GET	/api/faqs/search?q=keyword	Search FAQs by keyword	Public
PUT	/api/faqs/:id	Update an owned FAQ	Protected
DELETE	/api/faqs/:id	Delete an owned FAQ	Protected
POST	/api/ai/generate-faq	Generate an FAQ pair from a topic using Gemini	Protected

Sample Request Bodies

Register User

```
{"name":"Priya Sharma","email":"priya@writeflow.com","password":"securepassword123"}
```

Create FAQ

```
{"question":"How do I configure custom category tags?","answer":"Navigate to settings...","category":"Configuration"}
```

Generate FAQ

```
{"topic":"Mongoose schema indexing and validation"}
```

10. Authentication and Security

The application uses JWT-based authentication. After successful login, the server returns a signed token. Protected endpoints require the token in the HTTP Authorization header.

```
Authorization: Bearer <token>
```

- Authentication middleware extracts and verifies the JWT.
- The authenticated user context is attached to the request.
- Passwords are hashed with bcrypt before being stored.
- Protected FAQ modifications are restricted according to the application's authorization rules.
- Environment variables are used for sensitive configuration such as database credentials and API keys.

11. Running the Application

- Ensure MongoDB is running and the .env file is correctly configured.
- Start the development server using the configured npm script.
- Confirm the server is listening on the configured port.
- Use Postman or Thunder Client to test the API endpoints.
- Typical local base URL: `http://localhost:5000` (or the configured PORT).

12. Scenario and User Flow

Stage	User / System Action	Result
1. Registration	User submits name, email, and password.	Account is created securely.
2. Login	User submits valid credentials.	JWT token is returned.
3. Authentication	Token is sent with protected requests.	Access is verified by middleware.
4. FAQ Management	User creates, searches, updates, or deletes permitted FAQs.	Knowledge base is maintained.
5. AI Generation	User submits a topic to the AI endpoint.	Gemini generates FAQ content.
6. Retrieval	User searches available FAQ content.	Relevant records are returned.

Business Scenario: A customer-support operations manager needs to reduce repetitive responses, organize knowledge articles, classify FAQ content, and make information easier to retrieve. The API provides a centralized backend for these activities while maintaining authentication and data validation.

13. Testing and Validation

Testing Area	Validation Performed	Expected Outcome
Unit Testing	Authentication, FAQ CRUD, search, and AI modules tested individually.	Individual components behave correctly.
Integration Testing	Routes, middleware, models, and MongoDB interactions verified.	Components work together correctly.
API Testing	Register, login, FAQ, search, and AI endpoints tested.	Valid requests return expected responses.
Security Testing	Protected routes and password hashing verified.	Unauthorized requests are rejected and passwords remain protected.

Overall Result: The tested application demonstrates the intended data flow, API functionality, validation, and protected access behavior.

14. Demo and Project Resources

Resource	Link / Location
API Testing Video	https://drive.google.com/file/d/1u_w1sigiOynhmqTDbj4NBm3CLIMunEWN/view?usp=sharing
Demo Link	https://drive.google.com/file/d/1ajAZGgOaowPdyW2x5otcsLI84NeomSYJ/view?usp=drive_link
Code Drive	https://drive.google.com/drive/folders/1aUqV5q25fE_iUPwWcLt02eJ-v-ytxpjz?usp=sharing
GitHub Repository	https://github.com/sanjushree206-sketch/ai-faq-assistant

15. Known Limitations

- The current implementation is a backend REST API and does not include a built-in frontend interface.
- AI generation depends on Google Gemini service availability and API quota.
- The current search implementation is primarily keyword-based; advanced vector search is not implemented.
- Advanced administrator dashboard capabilities are limited.
- The current schema does not provide complete multi-tenant or organization-level data isolation.

16. Future Enhancements

- Implement embedding-based vector search for improved semantic relevance.
- Develop a React-based dashboard for non-technical content creators.
- Introduce organization-level and multi-tenant FAQ isolation.
- Add analytics for search trends and frequently requested questions.
- Implement rate limiting, API-key management, and usage quotas.
- Add webhooks and notifications for newly published FAQ content.
- Support FAQ export to Markdown, PDF, and knowledge-base formats.
- Explore domain-specific AI models for improved answer quality.

17. Team and Submission Details

Item	Official Details
Program / Platform	SmartBridge / SkillWallet
Team ID	SWTID-2026-8114
Team Size	4
Team Leader	SANJU SHREE R
Team Member	KRITHIKA N
Team Member	KIRTHIGA R
Team Member	S. DEEPIKA
Project	AI FAQ Assistant API

This document has been structured and formatted for project submission and technical evaluation.